

1. Learning Objectives
2. Introduction to the RCSB Protein Data Bank (PDB)
3. Visualizing the HIV-1 protease structure
4. Introduction to Bio3D in R
5. Comparative structure analysis of Adenylate Kinase
6. Normal mode analysis [optional]

# Structural Bioinformatics (Pt. 1)

[Code ▼](#)

## Hands-on Lab Sheet

Barry Grant < <http://thegrantlab.org/teaching/> (<http://thegrantlab.org/teaching/>) >  
2026-02-10 (09:12:46 on Tue, Feb 10)

## 1. Learning Objectives

By the end of this lab, you will be able to:

- Navigate the PDB database and describe it's biased composition statistics.
- Use Mol\* to visualize protein-ligand interactions.
- Employ bio3d to read and analyze PDB structure data.
- Use Normal Mode Analysis to predict protein functional motions.
- Apply PCA to characterize protein conformational dynamics.

## 2. Introduction to the RCSB Protein Data Bank (PDB)

The PDB archive is the major repository of information about the 3D structures of large biological molecules, including proteins and nucleic acids. Understanding the shape of these molecules helps to understand how they work. This knowledge can be used to help deduce a structure's role in human health and disease, and in drug development. The structures in the PDB range from tiny proteins and bits of DNA or RNA to complex molecular machines like the ribosome composed of many chains of protein and RNA.

In the first section of this lab we will interact with the main US based PDB website (note there are also sites in Europe and Japan).

Visit: <http://www.rcsb.org/> (<http://www.rcsb.org/>) and answer the following questions

NOTE: The **“Analyze” > “PDB Statistics” > “by Experimental Method and Molecular Type”** on the PDB home page should allow you to determine most of these answers. If the statistics page is taking too long to update due to server load then you can obtain the Feb 2026 numbers here (<https://tinyurl.com/pdbstats26>).

## PDB statistics

Open RStudio and begin a new **class** project. If we have covered GitHub in a previous class then you should create this within your **GitHub tacked directory/folder** from that class. Make sure *“Create a git repository”* option is **NOT** ticked. This is because we want to use the same git repository as we used last day and not start a new one - if you are not sure what this means ask Barry now!

Next, open a new **Quarto document** (**File > New File > Quarto Document...**). As always, we will aim to have a rendered PDF report with working code by the end of this class!

Download this **CSV file** (<https://tinyurl.com/pdbstats26>) into your RStudio project and use it to answer the following questions. Note that this data was obtained from the RCSB PDB website on Feb 6th 2026 using their **“Analyze” > “PDB Statistics” > “by Experimental Method and Molecular Type”** tool.

- **Q1:** What percentage of structures in the PDB are solved by X-Ray and Electron Microscopy.
- **Q2:** What proportion of structures in the PDB are protein?
- **Q3:** Type HIV in the PDB website search box on the home page and determine how many HIV-1 protease structures are in the current PDB?

## The PDB format

Now download the *“PDB File”* for the HIV-1 protease structure with the PDB identifier **1HSG**. On the website you can *“Display”* the contents of this *“PDB format”* file.

Alternatively, you can examine the contents of your downloaded file in a suitable text editor or use the Terminal tab from within RStudio (or your favorite Terminal/Shell) and try the following command:

Hide

```
less ~/Downloads/1hsg.pdb
```

```
## (use 'q' to quit)
```

NOTE: When viewing the file stop when you come to the lines beginning with the word **"ATOM"**. We will discuss this ubiquitous PDB file format when you have got this far.

|      | Element | Amino<br>Acid | Chain | Sequence/Residue<br>Number | Coordinates |        |        | (etc.)     |
|------|---------|---------------|-------|----------------------------|-------------|--------|--------|------------|
|      |         |               |       |                            | X           | Y      | Z      |            |
| ATOM | 1       | N             | MET   | A                          | 1           | 19.353 | 41.547 | -3.887 ... |
| ATOM | 2       | CA            | MET   | A                          | 1           | 20.513 | 40.939 | -4.592 ... |
| ATOM | 3       | C             | MET   | A                          | 1           | 20.150 | 39.658 | -5.355 ... |
| ATOM | 4       | O             | MET   | A                          | 1           | 19.053 | 39.551 | -5.903 ... |
| ATOM | 5       | CB            | MET   | A                          | 1           | 21.642 | 40.678 | -3.592 ... |
| ATOM | 6       | CG            | MET   | A                          | 1           | 21.233 | 39.903 | -2.360 ... |
| ATOM | 7       | SD            | MET   | A                          | 1           | 22.533 | 39.928 | -1.113 ... |
| ATOM | 8       | CE            | MET   | A                          | 1           | 23.771 | 38.881 | -1.885 ... |
| ATOM | 9       | N             | ASP   | A                          | 2           | 21.068 | 38.694 | -5.390 ... |
| ATOM | 10      | CA            | ASP   | A                          | 2           | 20.856 | 37.440 | -6.117 ... |
| ATOM | 11      | C             | ASP   | A                          | 2           | 20.124 | 36.371 | -5.299 ... |
| ATOM | 12      | O             | ASP   | A                          | 2           | 20.680 | 35.818 | -4.351 ... |

Element position within amino acid

Protein Data Bank files (or PDB files) are the most common format for the distribution and storage of high-resolution biomolecular coordinate data. At their most basic, PDB coordinate files contain a list of all the atoms of one or more molecular structures. Each atom position is defined by its x, y, z coordinates in a conventional orthogonal coordinate system. Additional data, including listings of observed secondary structure elements, are also commonly (but not always) detailed in PDB files.

Molecular graphics programs such as Mol\* (<https://molstar.org/>), VMD (<https://www.ks.uiuc.edu/Research/vmd/>), PyMol (<https://pymol.org/2/>) and Chimera (<https://www.cgl.ucsf.edu/chimera/>) take these files and plot them in 3D with the ability to make simplified and stylized representations such as the one shown below:

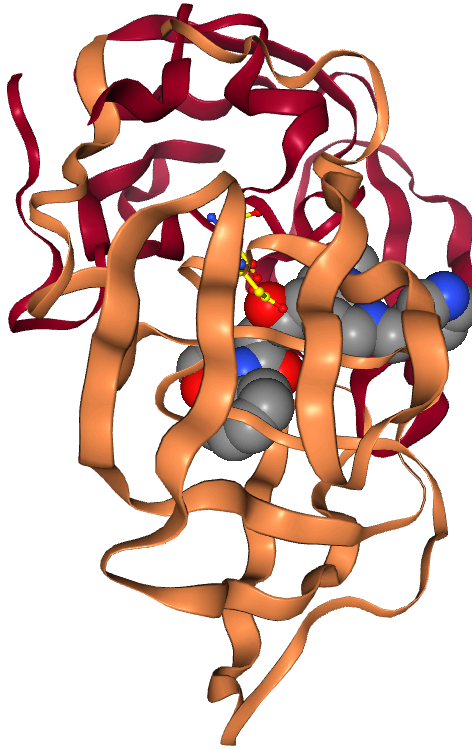


Figure 1. HIV-1 protease structure (PDB code: 1HSG (<https://www.rcsb.org/structure/1HSG>)) in complex with the small molecule indinavir.

### 3. Visualizing the HIV-1 protease structure

The HIV-1 protease is an enzyme that is vital for the replication of HIV. It cleaves newly formed polypeptide chains at appropriate locations so that they form functional proteins. Hence, drugs that target this protein could be vital for suppressing viral replication. A handful of drugs - called HIV-1 protease inhibitors (saquinavir, ritonavir, indinavir, nelfinavir, etc.) - are currently commercially available that inhibit the function of this protein, by binding in the catalytic site that typically binds the polypeptide.

In this section we will use the 2Å resolution X-ray crystal structure of HIV-1 protease with a bound drug molecule indinavir (PDB ID: 1HSG (<https://www.rcsb.org/structure/1hsg>)). We will use the Mol\* molecular viewer to visually inspect the protein, the binding site and the drug molecule. After exploring features of the complex we will move on to perform bioinformatics analysis of single and multiple crystallographic structures to explore the conformational dynamics and flexibility of the protein - important for its function and for considering during drug design.



# Using Mol\*

Mol\* (pronounced “molstar”) is a new web-based molecular viewer that is rapidly gaining in popularity and utility. At the time of writing it is still a long way from having the full feature set of stand-alone molecular viewer programs like VMD, PyMol or Chimera. However, it is gaining new features all the time and does not require any download or complicated installation.

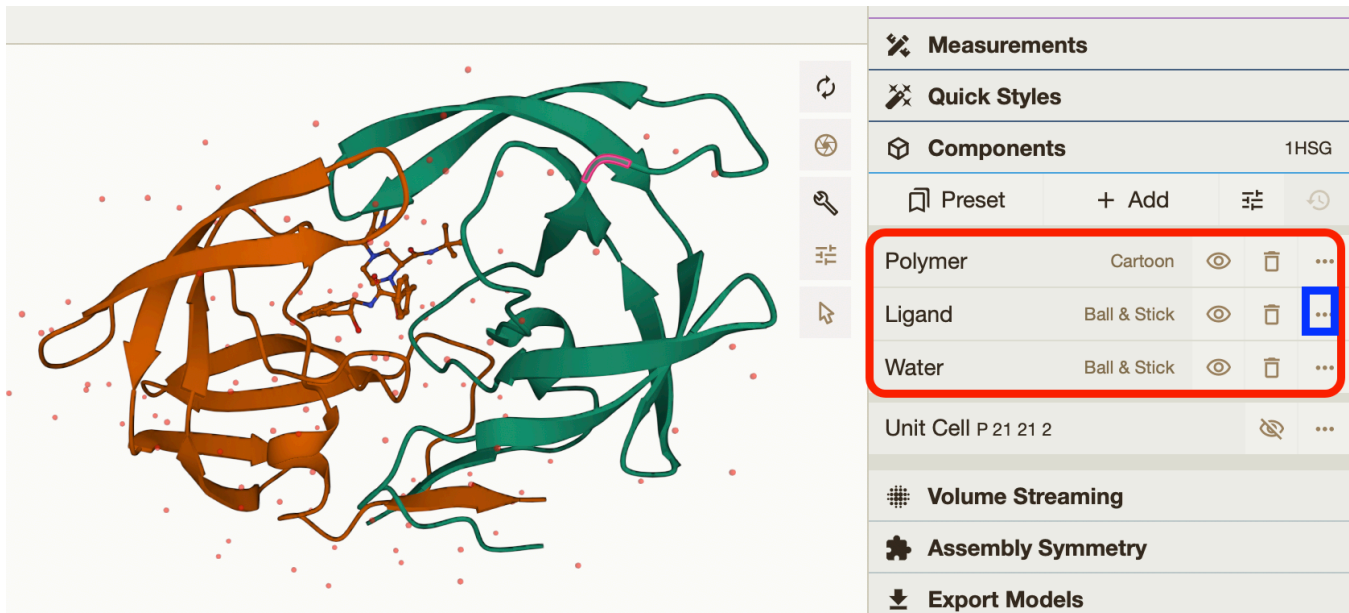
You can use Mol\* directly at the PDB website (as well as UniProt and elsewhere). However, for the latest and greatest version we will visit the Mol\* homepage at: <https://molstar.org/viewer/> (<https://molstar.org/viewer/>).

To load a structure from the PDB we can enter the *PDB code* and click “Apply” in the “Download Structure” menu (see figure below)



Once loaded the sidebar should change to the so-called hierarchical “*State Tree*” menu. Of particular note there are entries for **Polymer**, **Ligand** and **Water**. You can turn the display of any of these entries OFF/ON by clicking on the eye icon or delete them by clicking the “trash” bin icon (but we will not do that just yet). We can turn this left-side control panel off to save screen space. Especially as we will not need it again until we come to close the molecule or read a new molecule later.

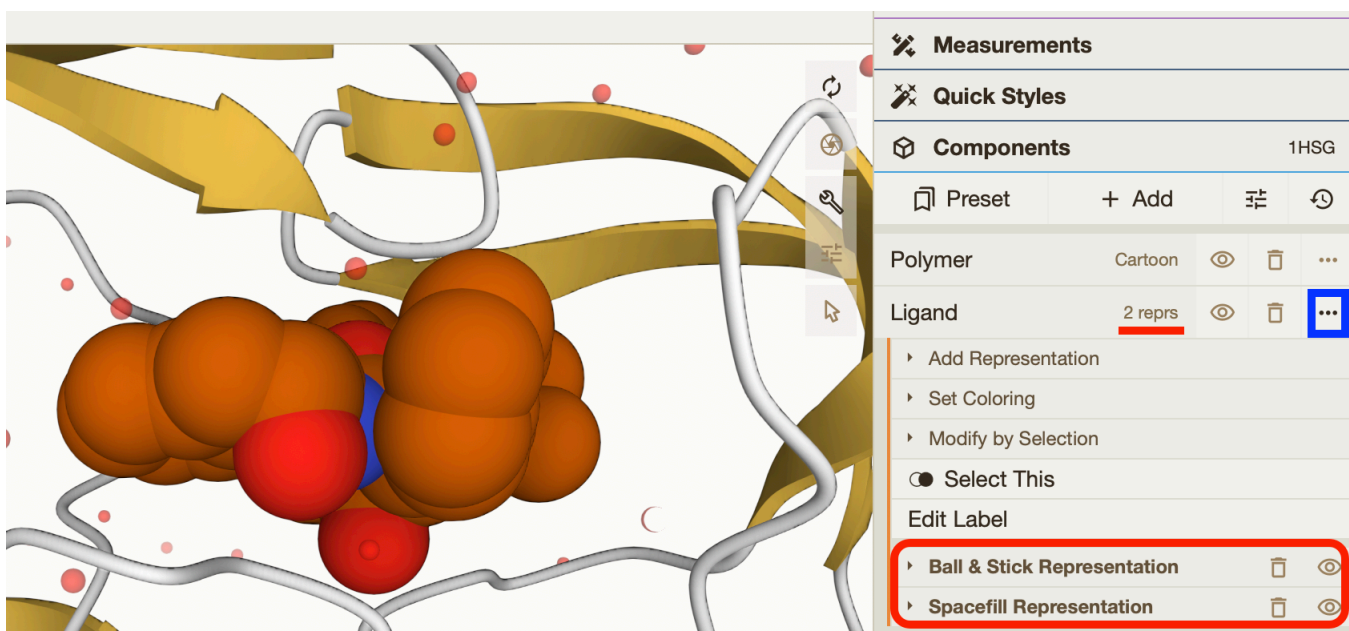
**Key-point:** You can access and change all visual representations on the opposite right side control panel under the “**Components**” drop-down menu (see figure below). Try toggling ON/OFF the display of Ligand and Water with the “eye” icon.



## Getting to know HIV-Pr

Let's temporarily toggle OFF/ON the display of **water** molecules and change the display representation of the **Ligand** to **Spacefill** (a.k.a VdW spheres). To do this:

- Click on **three dots** for the “Ligand” components entry in the right side control panel (blue box in above image),
- Then from the drop-down select **Add Representation > Spacefill**.
- Note that there are now two “*reps*” listed for the ligand component that you can control independently in the expandable menu accessible from clicking the three dots (see red box below).



Let's also change the protein "*Polymer*" > "**Set Coloring**" > "**Residue Property**" > "**Secondary Structure**".

**Key-point:** All these expanding drop-down menus can quickly become overwhelming. I find that closing them by clicking the 3 dots again can help keep things tidy and avoid menu items disappearing off small screens.

## Saving an image

Once you are happy with your display you can save a **high-resolution image** to your computer for including in your Quarto document. To do this find the "*iris-like*" screenshot icon on the right side of the display region and select your resolution and click download (see figure below)

### Screenshot / State





☒ **Auto-crop On**

 Copy

 **Download**

|             |                 |
|-------------|-----------------|
| Resolution  | HD (1280 x 720) |
| Transparent | × Off           |
| Axes        | × Off           |

► State

## Delving deeper

To help highlight important amino acid residues that interact with the ligand you can click on the ligand itself. This will lead to a new **“Focus Surroundings (5A)”** display component to appear. Mousing over this will highlight the corresponding amino acids in the Sequence display panel.

**Note:** Zoom in and rotate to examine these ligand interactions. Of these positions **Asp 25 (D25)** in both chains is critical for protease activity. Can you find this amino acid in both chains? Note the residue information displayed in the bottom right of the viewing window as you mouse over different amino acids.

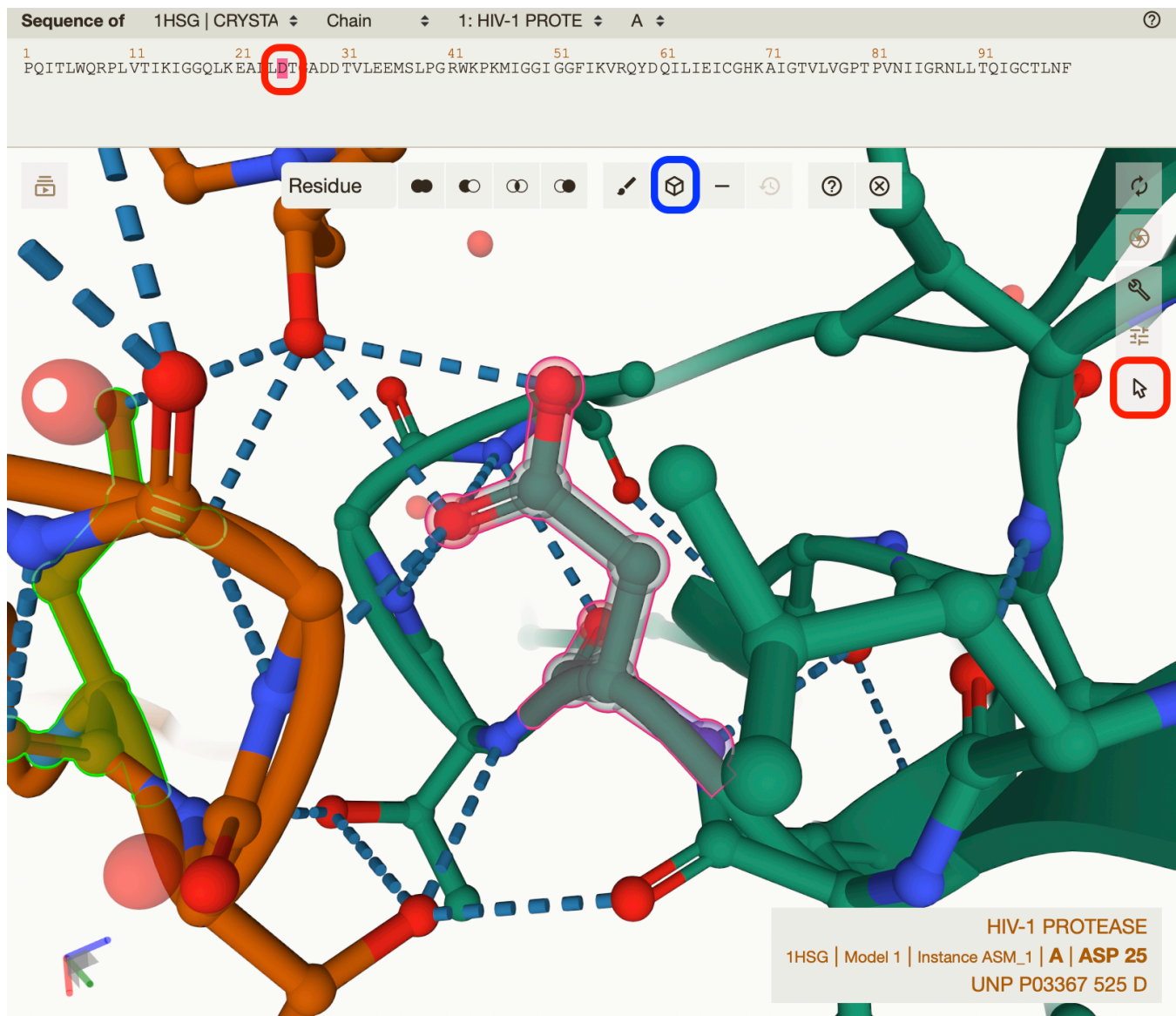
## Cleaning up the display

Most viewers will find that displaying all ligand surrounding amino acids is too busy for a single display. Turn off the display of these positions by clicking the eye” icon for the **“Focus Surroundings (5A)”** Components entry in the right side control panel.

Now we can highlight a subset of the most important positions:

- Using the top *Sequence display* select position **Asp 25 (D25)** in one of the chains.
- Now activate so-called **“Selection Mode”** by clicking the *Arrow icon* (red box to the right side of the 3D viewer panel in the figure below).
- Then select the two Asp 25 positions in the 3D structure.
- Finally click the **cube icon** (blue box in below figure) and from the drop-down menu that appears select *Representation* Spacefill or Ball & Stick (whatever you prefer), then click **+Create Component**.

Note that a new **“Custom Selection”** component has appeared in the right side control panel. This will contain your two D25 positions. You can again delete the **“Focus Surroundings (5A)”** and **Focus Target** Components to clean up the display.



At this point you should consider saving an image as discussed above.

## The important role of water

Toggle on the display of all water molecules again.

**Q4:** Water molecules normally have 3 atoms. Why do we see just one atom per water molecule in this structure?

**Q5:** There is a critical “conserved” water molecule in the binding site. Can you identify this water molecule? What residue number does this water molecule have

Now you should be able to produce an image similar or even superior to **Figure 2** and save it to an image file.



**Q6:** Generate and save a figure clearly showing the two distinct chains of HIV-protease along with the ligand. You might also consider showing the catalytic residues ASP 25 in each chain and the critical water (we recommend “*Ball & Stick*” for these side-chains). Add this figure to your Quarto document.

**Discussion Topic:** Can you think of a way in which indinavir, or even larger ligands and substrates, could enter the binding site?

**Q7:** [Optional] As you have hopefully observed HIV protease is a homodimer (i.e. it is composed of two identical chains). With the aid of the graphic display can you identify secondary structure elements that are likely to only form in the dimer rather than the monomer?

## 4. Introduction to Bio3D in R

Bio3D (<http://thegrantlab.org/bio3d/index.php>) is an R package for structural bioinformatics. Features include the ability to read, write and analyze biomolecular structure, sequence and dynamic trajectory data.

In your existing Rmarkdown document load the Bio3D package by typing in a new code chunk:

Hide

```
library(bio3d)
```

**Side-Note:** If you see an error message reported then you will first need to install the package with the command: `install.packages("bio3d")` in your **R Console** (i.e. don't put this in your Rmarkdown document or it will be re-installed every time you knit/render your document). This is only required once whereas the `library(bio3d)` command is required at the start of every new R session where you want to use Bio3D.

## Reading PDB file data into R

To read a single PDB file with Bio3D we can use the `read.pdb()` function. The minimal input required for this function is a specification of the file to be read. This can be either the file name of a local file on disc, or the RCSB PDB identifier of a file to read directly from the on-line PDB repository. For example to read and inspect the on-line file with PDB ID 1HSG:

Hide

```
pdb <- read.pdb("1hsg")
```

```
## Note: Accessing on-line PDB file
```

To get a quick summary of the contents of the `pdb` object you just created you can issue the command `print(pdb)` or simply type `pdb` (which is equivalent in this case):

[Hide](#)

```
pdb
```

```
##
## Call: read.pdb(file = "1hsg")
##
## Total Models#: 1
## Total Atoms#: 1686, XYZs#: 5058 Chains#: 2 (values: A B)
##
## Protein Atoms#: 1514 (residues/Calpha atoms#: 198)
## Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)
##
## Non-protein/nucleic Atoms#: 172 (residues: 128)
## Non-protein/nucleic resid values: [ HOH (127), MK1 (1) ]
##
## Protein sequence:
## PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYD
## QILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFPQITLWQRPLVTIKIGGQLKE
## ALLDTGADDTVLEEMSLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTP
## VNIIGRNLLTQIGCTLNF
##
## + attr: atom, xyz, seqres, helix, sheet,
## calpha, remark, call
```

- **Q7:** How many amino acid residues are there in this `pdb` object?
- **Q8:** Name one of the two non-protein residues?
- **Q9:** How many protein chains are in this structure?

Note that the attributes (`+ attr:`) of this object are listed on the last couple of lines. To find the attributes of any such object you can use:

[Hide](#)

```
attributes(pdb)
```



```
## $names
## [1] "atom"    "xyz"     "seqres"  "helix"   "sheet"   "calpha"  "remark"  "call"
##
## $class
## [1] "pdb"    "sse"
```

To access these individual attributes we use the `dollar-attribute` name convention that is common with R list objects. For example, to access the `atom` attribute or component use `pdb$atom`:

[Hide](#)

```
head(pdb$atom)
```

## Quick PDB visualization in R

We can use the Bio3D partner package, **bio3dview** (<https://bioboot.github.io/bio3dview/>), to generate quick interactive molecular visualizations. To install the development version of `bio3dview` from GitHub, along with the related `NGLViewer` package use:

[Hide](#)

```
install.packages("remotes")
remotes::install_github("bioboot/bio3dview")
install.packages("NGLViewer")
```

Then load the respective packages and generate a quick NGL (webGL based) structure overview of a `bio3d` `pdb` class object with a number of simple defaults. The returned `NGLViewer` object can be further added to build custom interactive visualizations:

[Hide](#)

```
library(bio3dview)
library(NGLViewer)

view.pdb(pdb) |>
  setSpin()
```

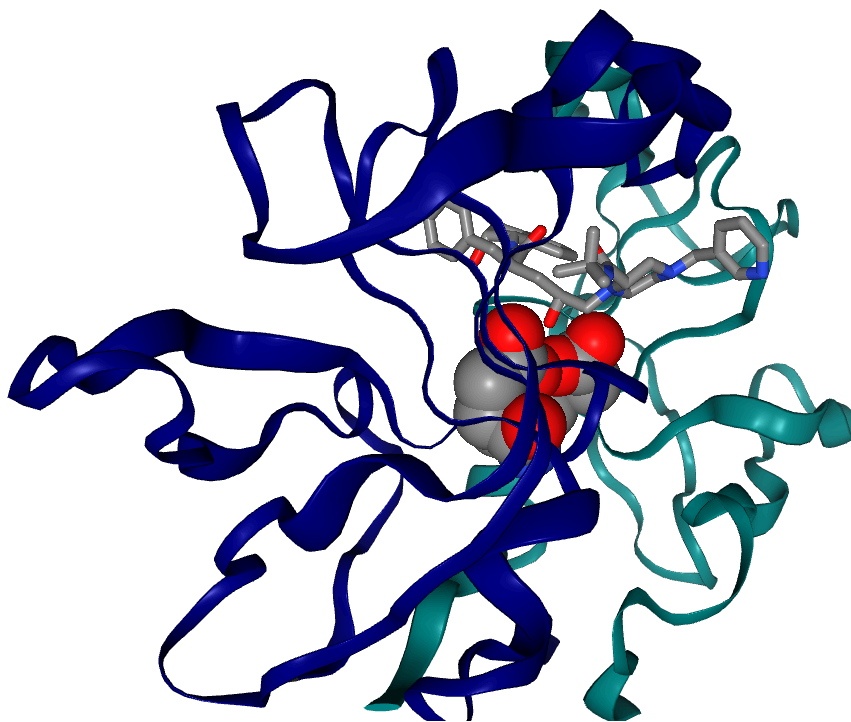


You can also customize the display in many ways with minimal code. For example, lets custom color the chains and highlight some key residues as spacefill/vdw:

[Hide](#)

```
# Select the important ASP 25 residue
sele <- atom.select(pdb, resno=25)

# and highlight them in spacefill representation
view.pdb(pdb, cols=c("navy","teal"),
          highlight = sele,
          highlight.style = "spacefill") |>
setRock()
```



## Predicting functional motions of a single structure

Let's read a new PDB structure of Adenylate Kinase and perform Normal mode analysis.

Hide

```
adk <- read.pdb("6s36")
```

```
## Note: Accessing on-line PDB file  
## PDB has ALT records, taking A only, rm.alt=TRUE
```

Hide

```
adk
```

```
##
## Call: read.pdb(file = "6s36")
##
## Total Models#: 1
## Total Atoms#: 1898, XYZs#: 5694 Chains#: 1 (values: A)
##
## Protein Atoms#: 1654 (residues/Calpha atoms#: 214)
## Nucleic acid Atoms#: 0 (residues/phosphate atoms#: 0)
##
## Non-protein/nucleic Atoms#: 244 (residues: 244)
## Non-protein/nucleic resid values: [ CL (3), HOH (238), MG (2), NA (1)
]
##
## Protein sequence:
## MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLRAAVKSGSELGKQAKDIMDAGKLV
## DELVIALVKERIAQEDCRNGFLLDGFRTIPQADAMKEAGINVDYVLEFDVPDELIVDKI
## VGRRVHAPSGRVYHVKNPPKVEGKDDVTGEELTRKDDQEETVRKRLVEYHQMTAPLIG
## YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
##
## + attr: atom, xyz, seqres, helix, sheet,
## calpha, remark, call
```

Normal mode analysis (NMA) is a structural bioinformatics method to predict protein flexibility and potential functional motions (a.k.a. conformational changes).

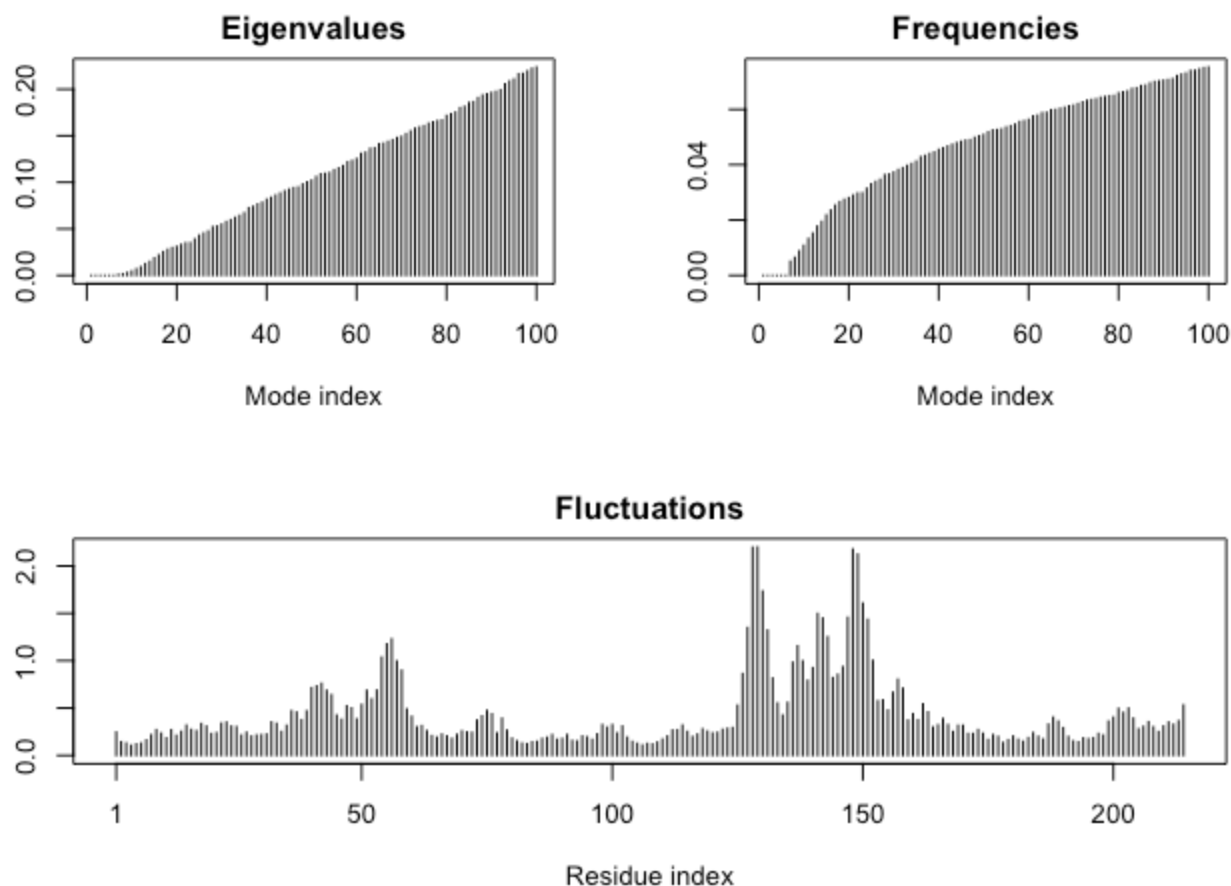
[Hide](#)

```
# Perform flexibility prediction
m <- nma(adk)
```

```
## Building Hessian... Done in 0.01 seconds.
## Diagonalizing Hessian... Done in 0.178 seconds.
```

[Hide](#)

```
plot(m)
```



To view a “movie” of these predicted motions we can generate a molecular “trajectory” with the `mktrj()` function.

Hide

```
mktrj(m, file="adk_m7.pdb")
```

Now we can load the resulting “*adk\_m7.pdb*” PDB into Mol\* with the “**Open Files**” option on the right side control panel. Once loaded click the “play” button to see a movie (see image below). We will discuss how this method works at the end of this lab when we apply it across a large set of homologous structures.



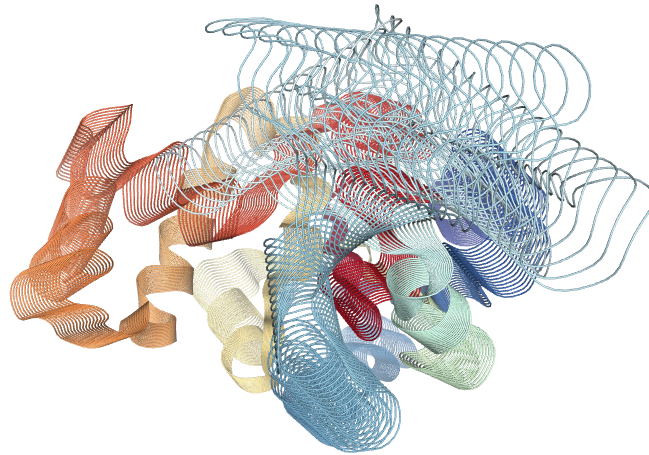
Here is what the output movie looks like:



Alternatively, for a quicker display you can use the `view.nma()` function from the `bio3dview` package mentioned previously:

[Hide](#)

```
view.nma(m, pdb=adk)
```



## 5. Comparative structure analysis of Adenylate Kinase

The goal of this section is to perform **principal component analysis** (PCA) on the complete collection of Adenylate kinase structures in the protein data-bank (PDB).

**Adenylate kinase** (often called simply Adk) is a ubiquitous enzyme that functions to maintain the equilibrium between cytoplasmic nucleotides essential for many cellular processes. Adk operates by catalyzing the reversible transfer of a phosphoryl group from ATP to AMP. This reaction requires a rate limiting conformational transition (i.e. change in shape). Here we analyze all currently available Adk structures in the PDB to reveal detailed features and mechanistic principles of these essential shape changing transitions.



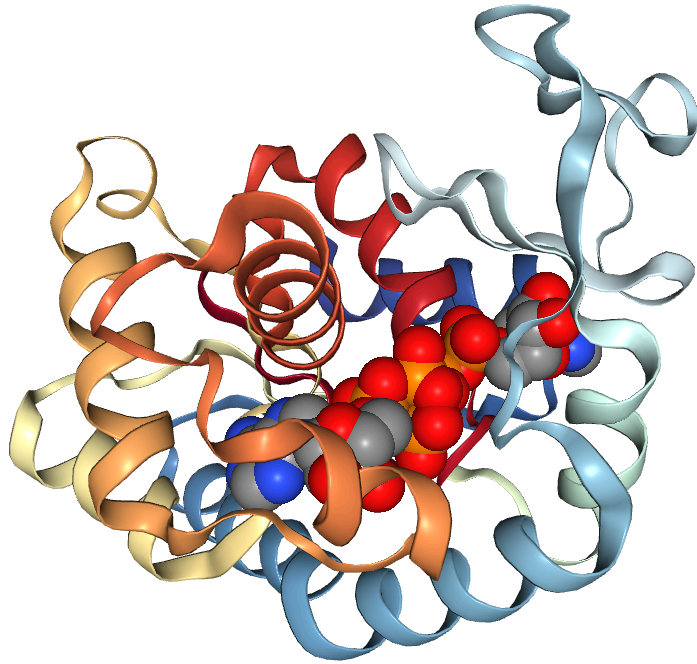


Figure 5. Adenylate kinase structure (PDB code: 1AKE (<https://www.rcsb.org/structure/1AKE>)) with a bound inhibitor molecule.

The **bio3d package** `pca()` function provides a convenient interface for performing PCA of biomolecular structure data. As we have discussed in previous classes, PCA is a statistical approach used to transform large data-sets down to a few important components that usefully describe the directions where there is most variance. In terms of protein structures PCA can be used to capture major structural variations within a set of structures (a.k.a. structure ensemble). This can make interpreting major conformational states (such as 'active' and 'inactive' or 'ligand bound' and 'un-bound' states) and structural mechanisms for activation or regulation more clear.

## Overview

Starting from only one Adk PDB identifier (**PDB ID: 1AKE**) we will search the entire PDB for related structures using BLAST, fetch, align and superpose the identified structures, perform PCA and finally calculate the normal modes of each individual structure in order to probe for potential differences in structural flexibility.

# Setup

We will begin by first installing the packages we need for today's session. Note that if you have followed along with the previous sections then you will already have all but the last of these (i.e. you will just need to install the **msa** package).

[Hide](#)

```
# Install packages in the R console NOT your Rmd/Quarto file

install.packages("bio3d")
install.packages("NGLViewer")

install.packages("remotes")
remotes::install_github("bioboot/bio3dview")

install.packages("BiocManager")
BiocManager::install("msa")
```

- **Q10.** Which of the packages above is found only on BioConductor and not CRAN?
- **Q11.** Which of the above packages is not found on BioConductor or CRAN?:
- **Q12.** True or False? Functions from the pak package can be used to install packages from GitHub and BitBucket? ☐ ☒

[More details](#)

## Search and retrieve ADK structures

Below we perform a blast search of the PDB database to identify related structures to our query Adenylate kinase (ADK) sequence. In this particular example we use function `get.seq()` to fetch the query sequence for chain A of the PDB ID 1AKE and use this as input to `blast.pdb()`. Note that `get.seq()` would also allow the corresponding UniProt identifier.

[Hide](#)

```
library(bio3d)
aa <- get.seq("1ake_A")
```

```
## Fetching... Please wait. Done.
```

Hide

aa

```

##          1          .          .          .          .          .          60
## pdb|1AKE|A  MRIILLGAPGAGKGTQAQFIMEKYGIPQISTGDMRLRAAVKSGSELGKQAKDIMDAGKLV
##          1          .          .          .          .          .          60
##
##          61          .          .          .          .          .          12
0
## pdb|1AKE|A  DELVIALVKERIAQEDCRNGFLLDGFRTIPQADAMKEAGINVDYVLEFDVPDELIVDRI
##          61          .          .          .          .          .          12
0
##
##          121         .          .          .          .          .          18
0
## pdb|1AKE|A  VGRRVHAPSGRVYHVKNPPKVEGKDDVTGEELTRKDDQEETVRKRLVEYHQMTAPLIG
##          121         .          .          .          .          .          18
0
##
##          181         .          .          .          214
## pdb|1AKE|A  YYSKEAEAGNTKYAKVDGTPVAEVRADLEKILG
##          181         .          .          .          214
##
## Call:
##   read.fasta(file = outfile)
##
## Class:
##   fasta
##
## Alignment dimensions:
##   1 sequence rows; 214 position columns (214 non-gap, 0 gap)
##
## + attr: id, ali, call

```

- **Q13.** How many amino acids are in this sequence, i.e. how long is this sequence?

### Optional:

Now we can use this sequence as a query to BLAST search the PDB to find similar sequences and structures.

Hide

```
# Blast or hmmer search
#b <- blast.pdb(aa)
```

**Side-note:** Due to the number of students in this class session this command, which uses online NCBI blast service, may time-out. If this happens please jump ahead to the next *Side-note* below to skip running the actual blast search.

The function `plot.blast()` facilitates the visualization and filtering of the Blast results. It will attempt to set a seed position to the point of largest drop-off in normalized scores (i.e. the biggest jump in E-values). In this particular case we specify a cutoff (after initial plotting) of to include only the relevant E.coli structures:

[Hide](#)

```
# Plot a summary of search results
#hits <- plot(b)
```

Figure 6: Blast results. Visualize and filter blast results through function `plot.blast()`. Here we proceed with only the top scoring hits (black).

[Hide](#)

```
# List out some 'top hits'
#head(hits$ pdb.id)
```

## Required:

**Side-note:** If blast did not return results (likely due to the large number of simultaneous requests from the class) you can use the following vector of PDB IDs

[Hide](#)

```
hits <- NULL
hits$ pdb.id <- c('1AKE_A', '6S36_A', '6RZE_A', '3HPR_A', '1E4V_A', '5EJE_A', '1E4Y_A', '3X2S_A', '6HAP_A', '6HAM_A', '4K46_A', '3GMT_A', '4PZL_A')
```

The Blast search and subsequent filtering identified a total of 13 related PDB structures to our query sequence. The PDB identifiers of this collection are accessible through the `$pdb.id` attribute to the `hits` object (i.e. `hits$ pdb.id`). Note that adjusting the cutoff argument (to `plot.blast()`) will result in a decrease or increase of hits.

We can now use function `get.pdb()` and `pdbslit()` to fetch and parse the identified structures.

[Hide](#)

```
# Download related PDB files
files <- get.pdb(hits$pdb.id, path="pdbs", split=TRUE, gzip=TRUE)
```

## Align and superpose structures

Next we will use the `pdbaln()` function to align and also optionally fit (i.e. superpose) the identified PDB structures.

[Hide](#)

```
# Align related PDBs
pdbs <- pdbaln(files, fit = TRUE, exefile="msa")
```

Note that if you get an error msg above include the optional `exefile="msa"` input argument to `pdbaln()`.

## Optional: Viewing our superposed structures

We can view our superposed results with the new **bio3dview** `view()` function:

[Hide](#)

```
library(bio3dview)

view.pdbs(pdbs)
```

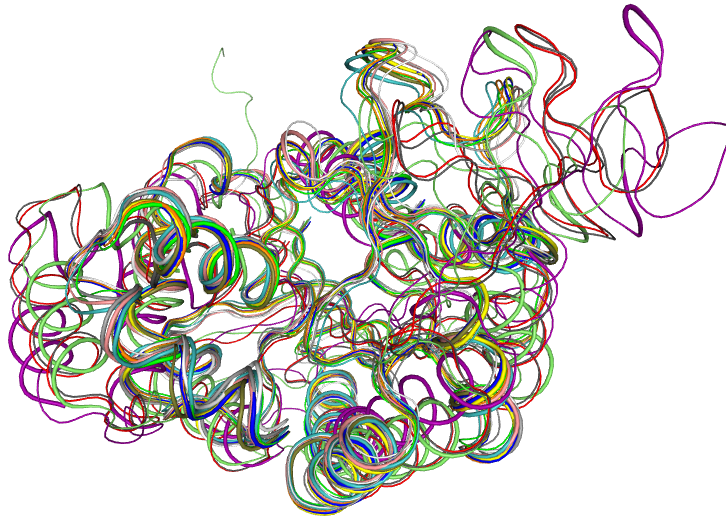


Figure 8: 3D view of superposed ADK structures available in the PDB.

**Tip:** Try setting the `colorScheme=residueIndex` argument to more clearly see the regions of structure with the greatest differences

(e.g. `view.pdbs(pdbs, colorScheme = "residueIndex")`)

## Annotate collected PDB structures

The function `pdb.annotate()` provides a convenient way of annotating the PDB files we have collected. Below we use the function to annotate each structure to its source species. This will come in handy when annotating plots later on:

Hide

```
# Vector containing PDB database codes
ids <- basename.pdb(pdbs$id)

anno <- pdb.annotate(ids)
unique(anno$source)
```

```
## [1] "Escherichia coli"  
## [2] "Escherichia coli K-12"  
## [3] "Escherichia coli 0139:H28 str. E24377A"  
## [4] "Escherichia coli str. K-12 substr. MDS42"  
## [5] "Photobacterium profundum"  
## [6] "Burkholderia pseudomallei 1710b"  
## [7] "Francisella tularensis subsp. tularensis SCHU S4"
```

We can view all available annotation data:

[Hide](#)

```
anno
```

## Principal component analysis

Function `pca()` provides principal component analysis (PCA) of the structure data. PCA is a statistical approach used to transform a data set down to a few important components that describe the directions where there is most variance. In terms of protein structures PCA is used to capture major structural variations within an ensemble of structures.

PCA can be performed on the structural ensemble (stored in the `pdb`s object) with the function `pca.xyz()`, or more simply `pca()`.

[Hide](#)

```
# Perform PCA  
pc.xray <- pca(pdb)s  
plot(pc.xray)
```

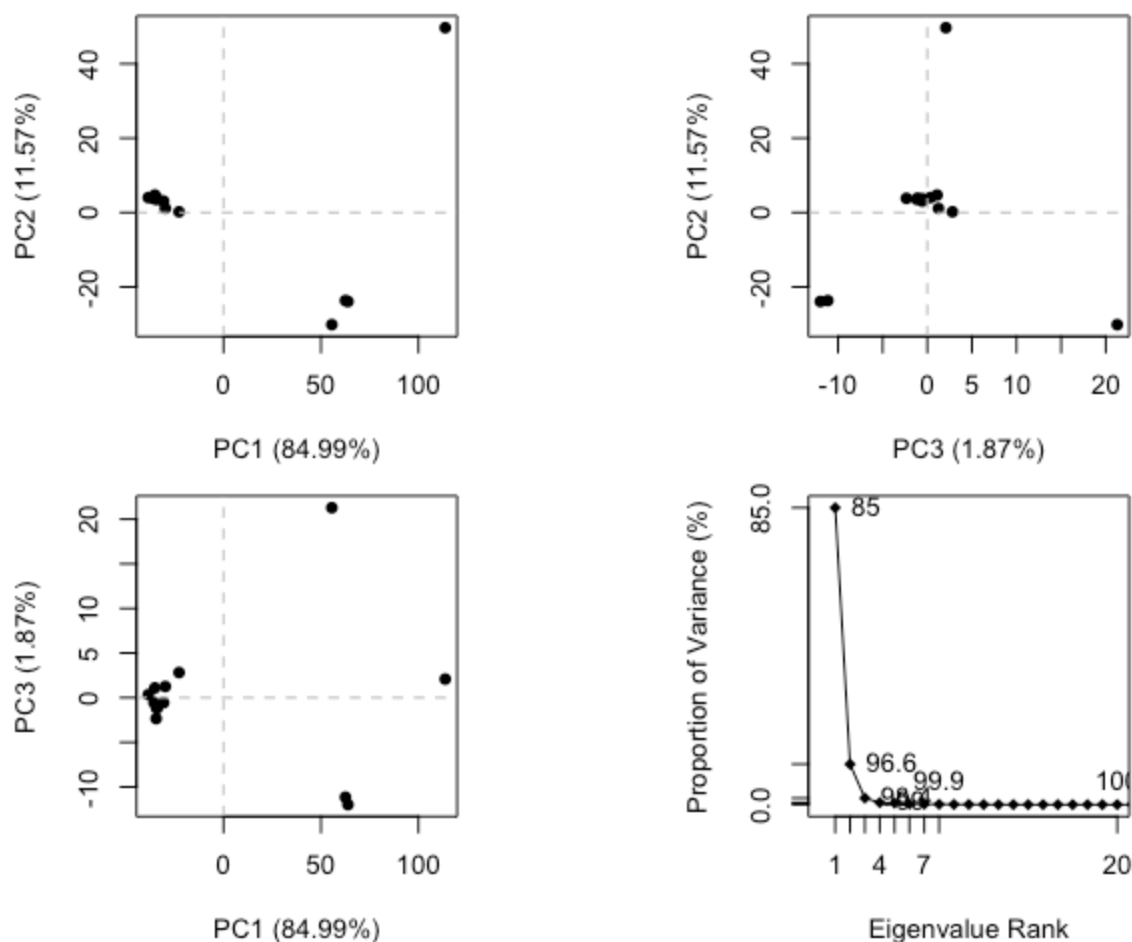


Figure 9: Results of PCA on Adenylate kinase X-ray structures. Each dot represents one PDB structure. Function `rmsd()` will calculate all pairwise RMSD values of the structural ensemble. This facilitates clustering analysis based on the pairwise structural deviation:

Hide

```
# Calculate RMSD
rd <- rmsd(pdb)
```

*# Structure-based clustering*

```
hc.rd <- hclust(dist(rd))
grps.rd <- cutree(hc.rd, k=3)
```

```
plot(pc.xray, 1:2, col="grey50", bg=grps.rd, pch=21, cex=1)
```



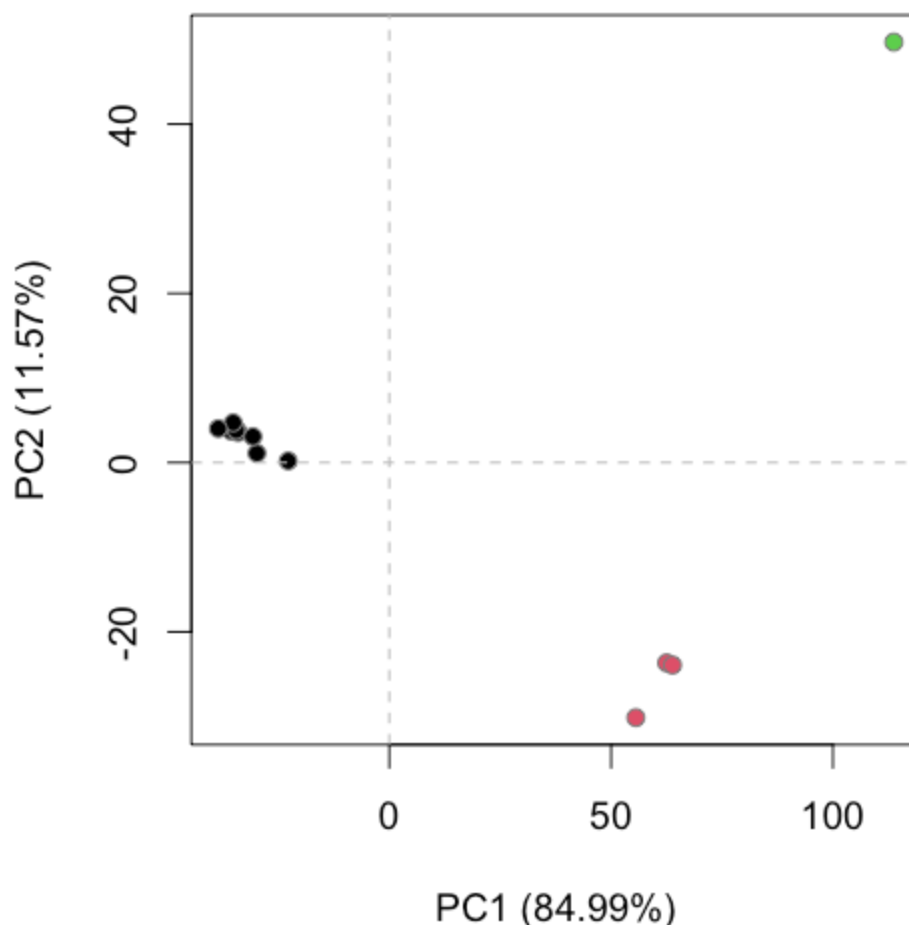


Figure 10: Projection of Adenylate kinase X-ray structures. Each dot represents one PDB structure.

The plot shows a conformer plot – a low-dimensional representation of the conformational variability within the ensemble of PDB structures. The plot is obtained by projecting the individual structures onto two selected PCs (e.g. PC-1 and PC-2). These projections display the inter-conformer relationship in terms of the conformational differences described by the selected PCs.

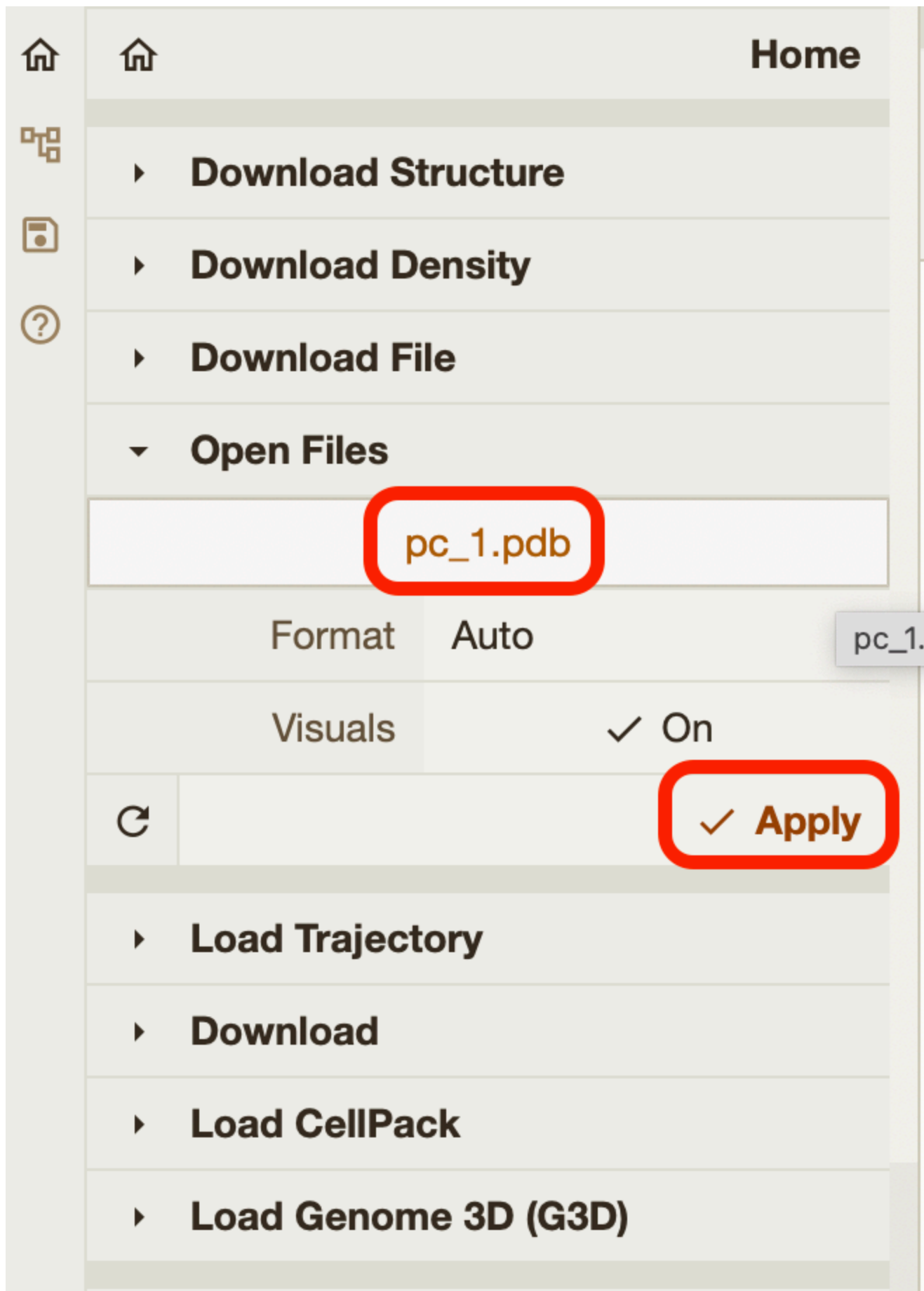
## PCA visualization

To visualize the major structural variations in the ensemble the function `mktrj()` can be used to generate a trajectory PDB file by interpolating along a give PC (eigenvector):

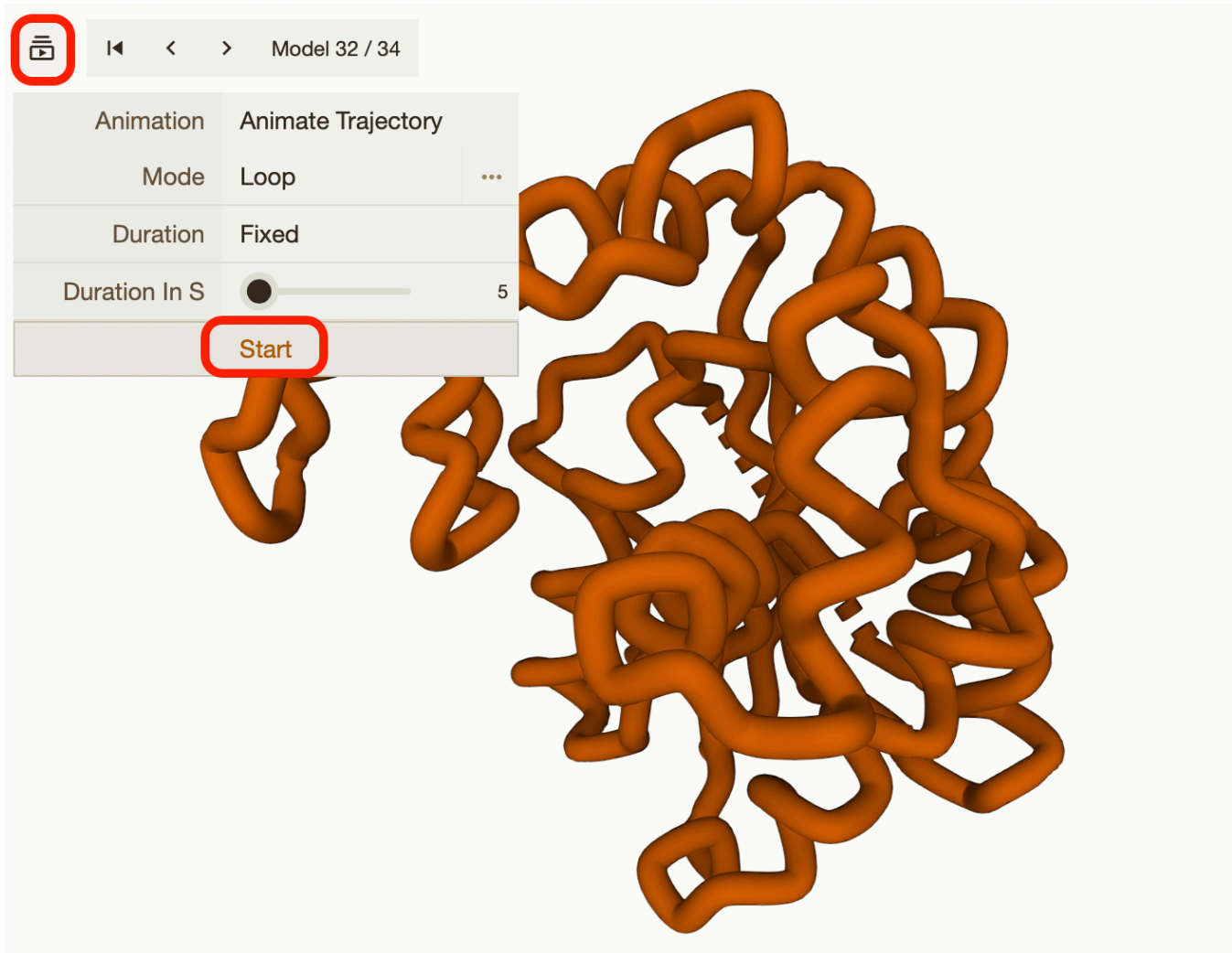
Hide

```
# Visualize first principal component  
pc1 <- mktrj(pc.xray, pc=1, file="pc_1.pdb")
```

You can open this file, `pc_1.pdb`, in Mol\*. In a web browser page visit <https://molstar.org/viewer/> (<https://molstar.org/viewer/>) and “**Open Files**” from the left control panel selecting .



Once loaded you can animate the structure and visualize the major structural variations along PC1 by clicking the “Play” icon and *Start* button (see below).



You can also save a movie of this motion via the “**Export Animation**” menu option on the right control panel:

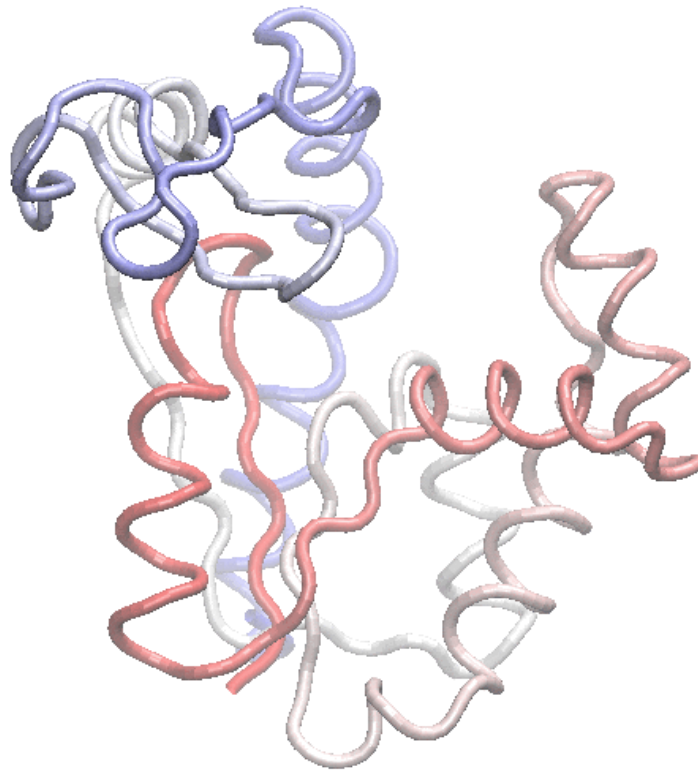


Figure 11: Visualization of PC-1 in VMD. Trajectory PDB file is generated using `mkt rj ( )` .

Skip

We can also plot our main PCA results with **ggplot**:

Hide

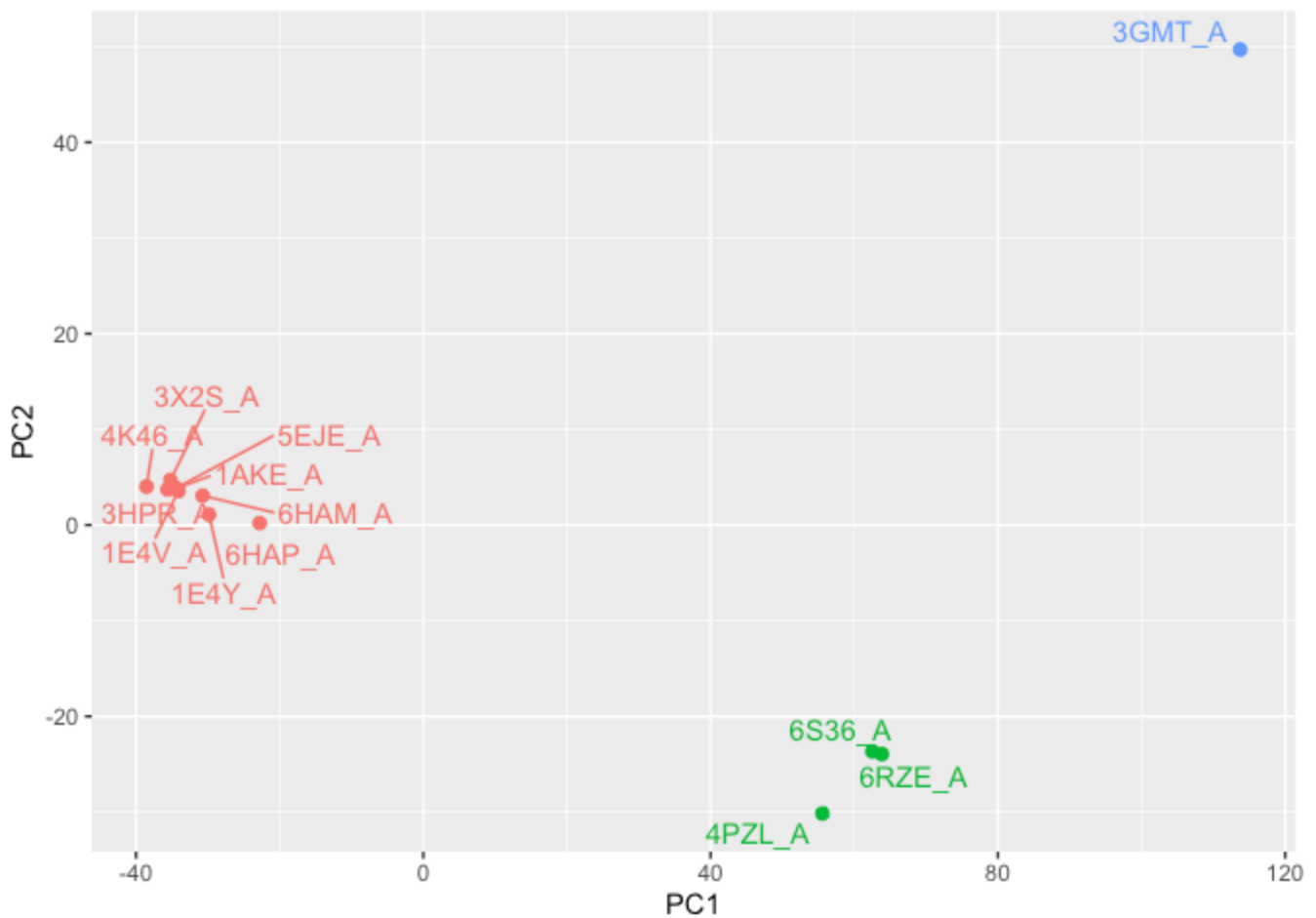
```

#Plotting results with ggplot2
library(ggplot2)
library(ggrepel)

df <- data.frame(PC1=pc.xray$z[,1],
                  PC2=pc.xray$z[,2],
                  col=as.factor(grps.rd),
                  ids=ids)

p <- ggplot(df) +
  aes(PC1, PC2, col=col, label=ids) +
  geom_point(size=2) +
  geom_text_repel(max.overlaps = 20) +
  theme(legend.position = "none")
p

```



## 6. Normal mode analysis [optional]

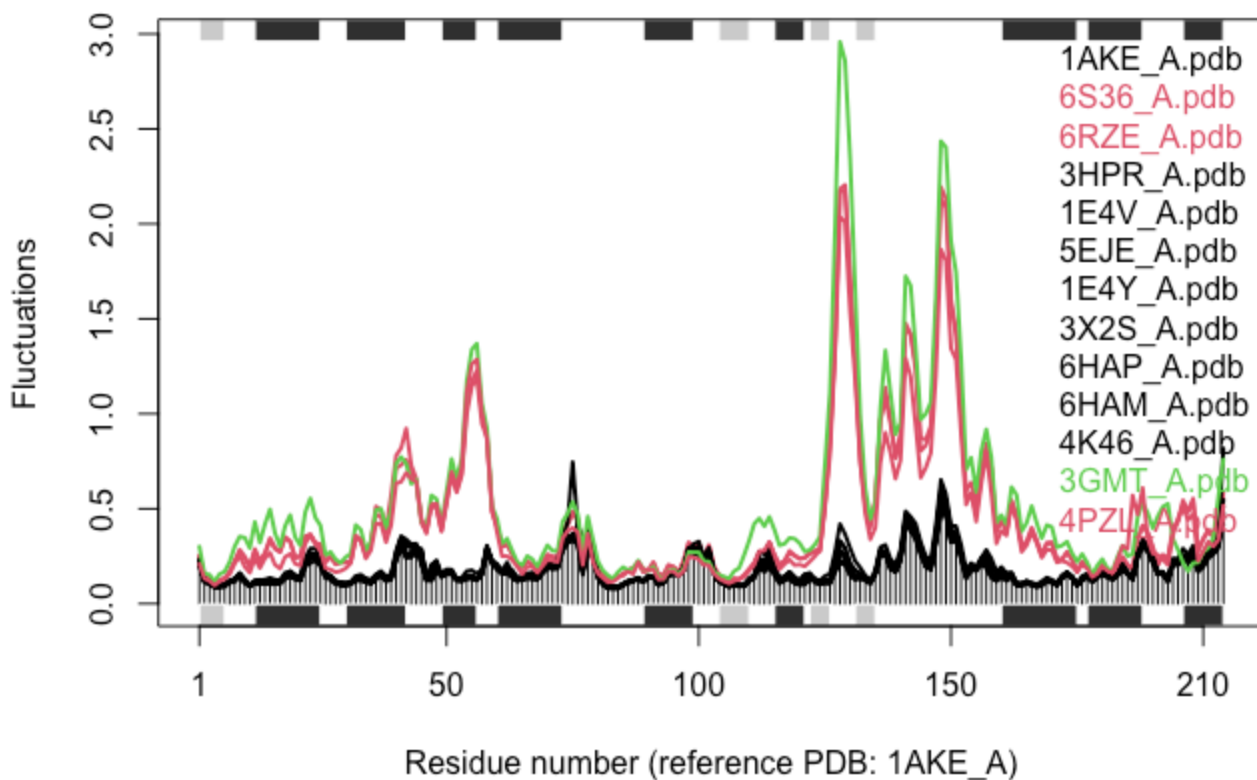
Function `nma()` provides normal mode analysis (NMA) on both single structures (if given a single PDB input object) or the complete structure ensemble (if provided with a PDBS input object). This facilitates characterizing and comparing flexibility profiles of related protein structures.

Hide

```
# NMA of all structures
modes <- nma(pdb)
```

Hide

```
plot(modes, pbs, col=grps.rd)
```



**Q14.** What do you note about this plot? Are the black and colored lines similar or different? Where do you think they differ most and why?

Collectively these results indicate the existence of two major distinct conformational states for Adk. These differ by a collective low frequency displacement of two nucleotide-binding site regions that display distinct flexibilities upon nucleotide binding.

**Important-Note:** Remember to save your Quarto document and Render to generate a HTML and PDF report for GradeScope.

About this document

---

Powered by labsheet (<https://github.com/bioboot/labsheet/>)

